Session 4 : Containers vs Virtual Machines

RECAP: Session 3 : Virtualization & Hypervisors

1. Virtualization Basics: Physical vs virtual resources; consolidation, isolation & utilization; types: server, storage, network, desktop, application
2. Hypervisors: Type 1 (bare-metal) vs Type 2 (hosted); Virtual Machine Monitor (VMM) & its role
3. Virtualization Techniques: Full virtualization; para-virtualization; hardware-assisted virtualization (VT-x / AMD-V)
4. Virtual Machine Anatomy: vCPU, virtual memory, virtual disk & virtual NIC; machine images, templates & snapshots
5. VM Lifecycle & Migration: Provision, start, suspend, snapshot, clone & terminate; live migration & its uses

Agenda for Today's Session

1. Container Fundamentals: Process-level isolation; Linux namespaces & control groups (cgroups); union filesystem & layered images
2. Docker Architecture: Client, daemon, images, containers & registries; image vs container; layers & build caching
3. Containers vs VMs: Boot time, memory footprint, image size & density; isolation strength & security trade-offs
4. Choosing Between Them: Container use cases; VM use cases; decision criteria
5. Tooling & Registries: VirtualBox, KVM, Hyper-V & VMware; Docker, Podman & containerd; public vs private registries; tagging strategy

---------- SECTION 1: 1. Container Fundamentals: Process-level isolation; Linux namespaces & control groups (cgroups); union filesystem & layered images ----------

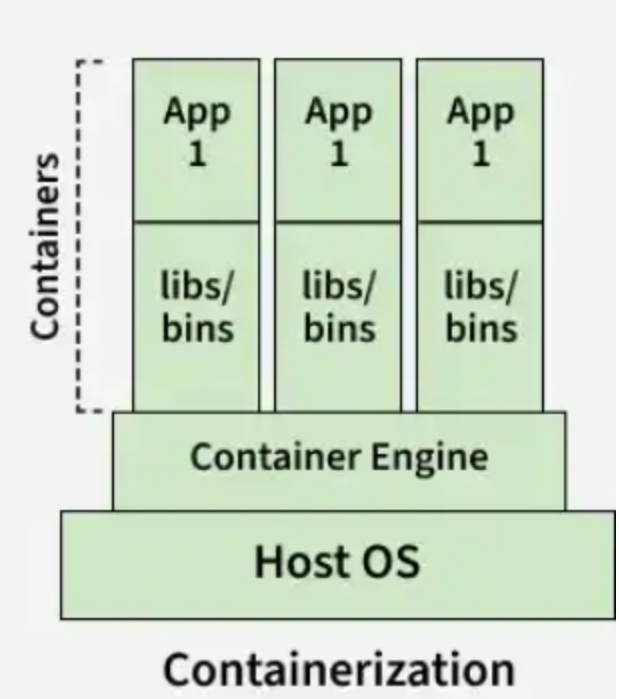
Slide 1: What Is A Container

Heading: Container Isolation Basics

Subheading: Process-Level Isolation

Content:

A container is a running process that shares the host machine's kernel but gets its own isolated view of files, network and other processes, with no separate guest operating system of its own.



- Namespaces : limit WHAT a process can see
- Control groups : limit HOW MUCH it can use
- Union filesystem : layers that build the image itself

Analogy: A large building holds many separate flats: each flat has its own rooms and lock, but all share one foundation, electricity and water line — exactly as containers share one host kernel while each gets its own isolated view through namespaces.

Visual Guidance: Single host kernel box at bottom; three container boxes on top, each labelled with a small lock icon and tagged Namespaces / cgroups / Union FS.

Speaker Notes: "Stress that isolation here is logical, enforced by the kernel, not a separate virtual hardware layer. Interviewers often ask what actually makes a container 'isolated' — the answer is these three kernel features, not a hypervisor."

Slide 2: Linux Namespaces Explained

Heading: Linux Namespaces

Subheading: Isolating Process Views

Content:

A namespace isolates a process's view of system resources, so each container gets its own view of processes, networking, and filesystems.

Namespace	Isolates
PID (process ID)	Own process tree, own PID 1
Network	Own interfaces, IP address, ports
Mount	Own filesystem view and mount points
UTS (UNIX time-sharing)	Own hostname and domain name
IPC (inter-process communication)	Own message queues, semaphores
User	Own user and group ID mapping

Visual Guidance: Six small labelled boxes around a central kernel icon, each box pointing to a different resource type from the table.

Speaker Notes: "A common interview trap is confusing PID namespace isolation with resource limiting — namespaces only change what is visible, not how much can be used. Mention that chroot was an early, narrower ancestor of the mount namespace idea."

Slide 3: Cgroups And Union Filesystem

Heading: Cgroups And Union FS

Subheading: Resource Limits Layers

Content:

Two more kernel features complete container isolation: control groups, known as cgroups, cap resource use, and a union filesystem builds the image itself from layers.

- CPU controller: caps processor time share per group
- Memory controller: breach triggers an out-of-memory kill
- Block I/O controller: caps disk read and write speed
- Union filesystem merges read-only layers; OverlayFS is the current standard driver
- Copy-on-write duplicates a file only when it is modified

Visual Guidance: One cgroup boundary box with CPU/Memory/Disk meter icons, beside a layered stack merging into one view with a writable top layer.

Speaker Notes: "A process crossing its memory ceiling is killed by the kernel, not slowed down. Layer caching from the union filesystem is why rebuilding an image after a small change is fast."

---------- SECTION 2: 2. Docker Architecture: Client, daemon, images, containers & registries; image vs container; layers & build caching ----------

Slide 4: Docker Architecture Overview

Heading: Docker Architecture Map

Subheading: Five Core Parts

Content:

Docker runs as a client-server system: a command-line client talks to a background daemon that builds and runs containers using images pulled from a registry.

- Docker client — CLI issuing commands like docker run
- Docker daemon (dockerd) — service building and managing containers
- Images — read-only templates stored locally or in a registry
- Containers — running instances created from images
- Registry — remote store such as Docker Hub for images

Visual Guidance: Diagram: client box with arrow to daemon box; daemon connects to an images store, a container box, and a registry cloud icon.

Speaker Notes: "Point out dockerd as the process name learners will see in ps output. Interviewers often ask to name all five Docker architecture pieces and how they interact."

Slide 5: Installing And Verifying Docker

Heading: Docker Installation Steps

Subheading: Setup Per OS

Content:

Docker installs differently per operating system, but every path ends with the same two verification commands before any practical work begins.

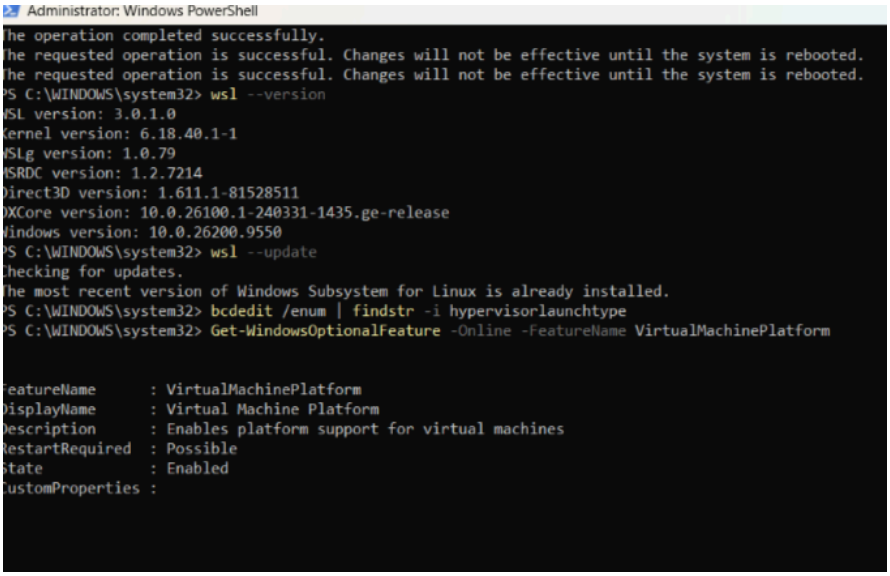
OS	Key Setup Steps
Windows	Install VS Code; wsl --install; Docker Desktop (WSL 2 backend); open Docker Desktop
macOS	Install VS Code; Docker Desktop (Apple Silicon or Intel build); open Docker Desktop
Linux / Ubuntu	Install VS Code; sudo apt install docker.io; sudo systemctl start docker

```
docker --version
docker run hello-world
```

- Line 1 — Displays the installed Docker CLI version.
- Line 2 — Verifies that Docker can run a container; it typically pulls the image if needed and prints Hello from Docker!

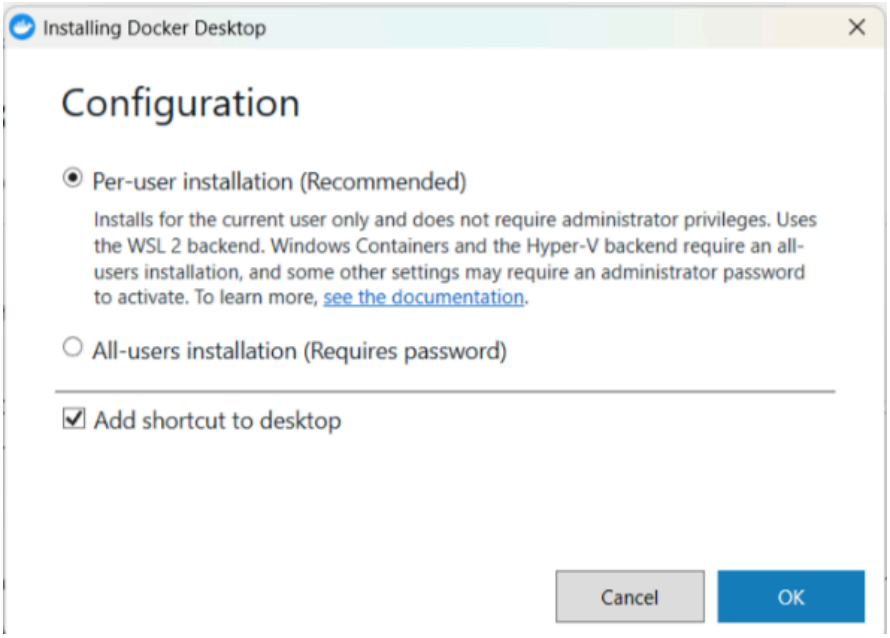
Linux OS: If a Docker command fails due to permission issues, try running it with sudo, for example, sudo docker run hello-world.

Step 1 — Confirm WSL 2 Is Enabled (Windows)



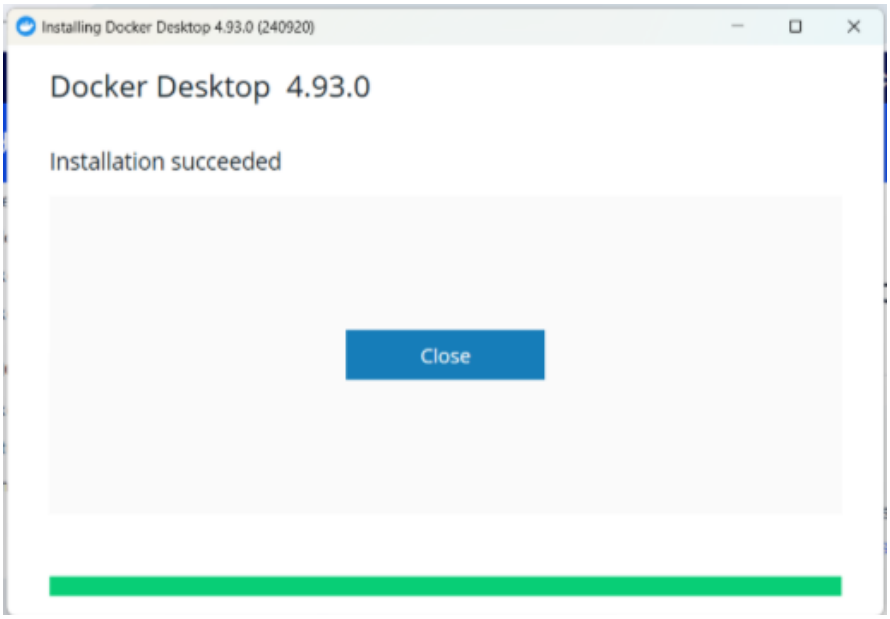
PowerShell confirms WSL is installed/updated and the Virtual Machine Platform feature is Enabled — required before Docker Desktop will run on Windows

Step 2 — Choose Installation Options



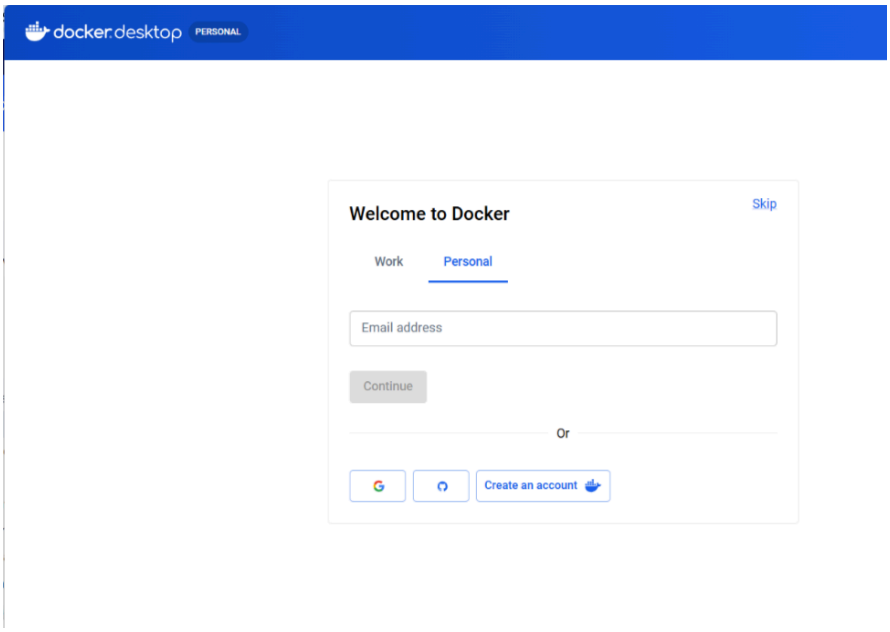
Docker Desktop installer — Configuration screen, per-user installation with the WSL 2 backend

Step 3 — Installation Completes



Docker Desktop installer confirms "Installation succeeded"

Step 4 — Sign In Or Skip



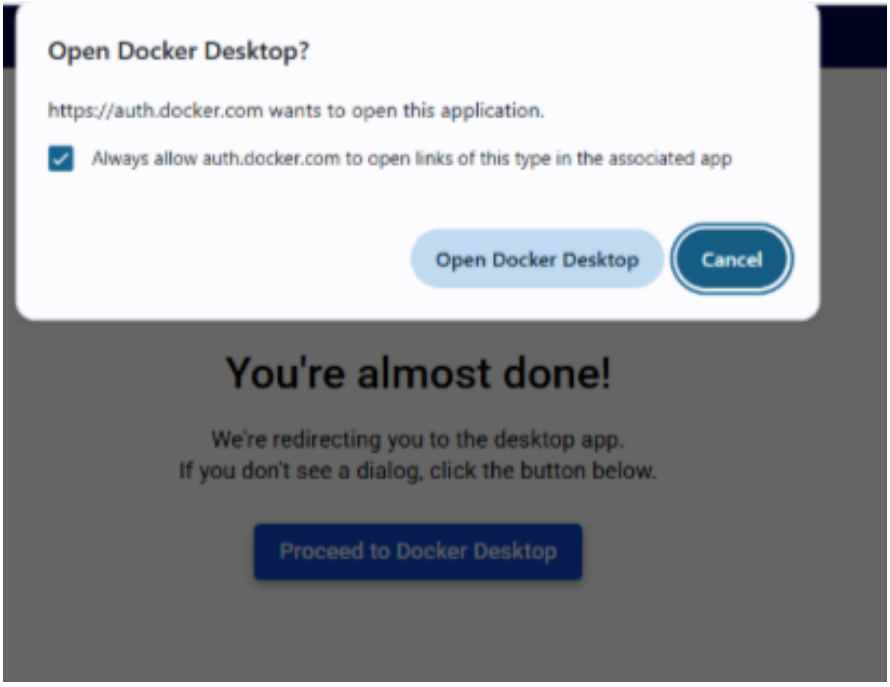
First launch — Docker prompts to sign in or create a Docker Hub account; signing in is optional for local use and can be skipped

Step 5 — Redirected To The Desktop App



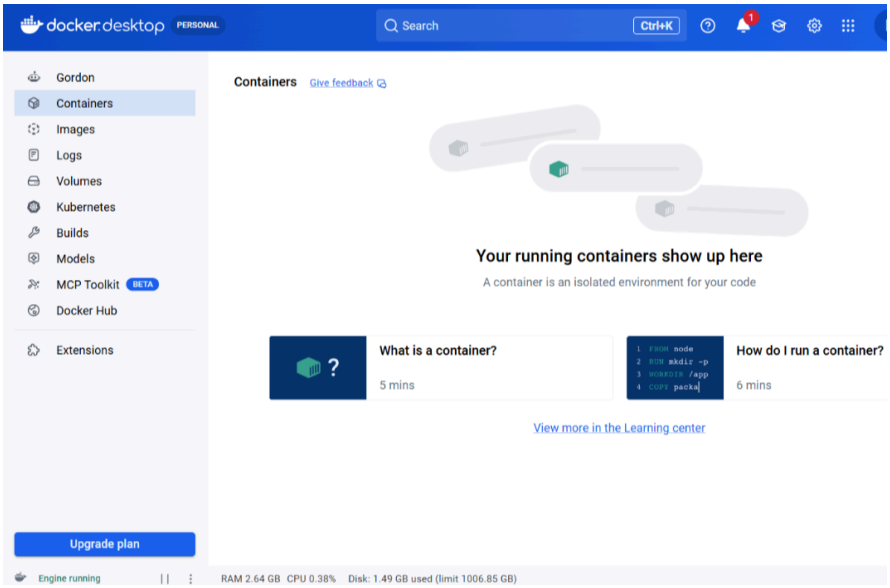
After sign-in (or skip), Docker redirects back to the desktop app

Step 6 — Confirm The Redirect



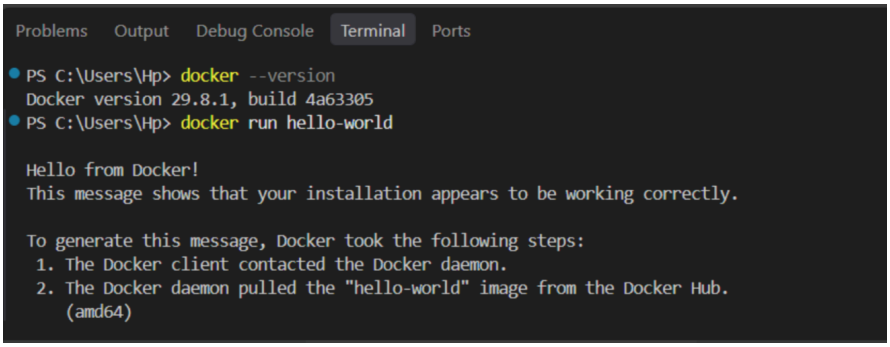
Windows asks permission to open Docker Desktop — confirms the handoff from browser back to the installed app

Step 7 — Docker Desktop Opens



Docker Desktop's Containers tab on first launch — empty, since nothing has been run yet

Step 8 — Verify From The Terminal



docker --version displays the installed Docker CLI version*; docker run hello-world confirms the daemon pulled and ran an image — "Hello from Docker!" confirms success*

Visual Guidance: See the 8-step screenshot walkthrough above — Windows install path shown end to end, from enabling WSL 2 through the docker run hello-world verification.

Speaker Notes: "Treat the final verification step as the gate before any practical work begins. A missing WSL 2 backend (Step 1) is the most common installation failure asked about. Docker Hub sign-in (Steps 4–6) is optional for local use — mention this so students don't think an account is mandatory."

Slide 6: Image vs Container Distinction

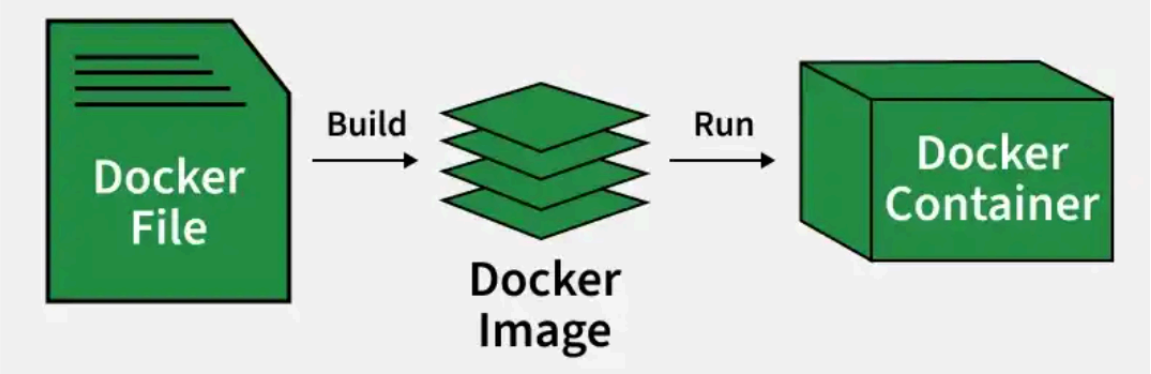
Heading: Image Vs Container

Subheading: Template Vs Instance

Content:

An image is a frozen, read-only template; a container is that template brought to life with its own writable layer on top.

- Image: built once, never changes, stored by ID and tag
- Container: created from an image, has a thin writable layer
- One image can start many independent containers
- Deleting a container never deletes its source image



Analogy: A cake recipe card is reused to bake many separate cakes — just as one Docker image can start many independent containers.

Visual Guidance: Side-by-side: image box with locked layers next to a container box showing the same layers plus a writable top layer.

Speaker Notes: "Tags like nginx:alpine name images, not containers. Interviews often ask what survives container removal — the image."

Slide 7: Pulling From A Registry

Heading: Client Daemon Registry

Subheading: Pull And Run Flow

Content:

Running an image that the daemon does not have locally triggers a pull from a registry — Docker Hub by default — before a container starts.

- Explicit pull: docker pull nginx:alpine fetches an image with no container created — the same mechanism docker run triggers automatically
- docker run hello-world: no flags — image name only, pulled if absent locally
- Client sends the request over the daemon's local API socket
- Daemon checks its local image cache first by name and tag
- Pulled layers are cached, so a repeat run skips the registry

Visual Guidance: Arrow sequence: client -> daemon -> registry (dashed, only on cache miss) -> container created.

Speaker Notes: "Explain that a one-shot container exits once its process finishes, which is normal. Interviewers often ask why a stopped container still appears in docker ps -a."

Slide 8: Part A Practical Steps

Heading: Part A Practical Steps

Subheading: Images To Containers

Content:

Part A works with an existing image, tracing every command from an explicit pull to removing a running web server container.

- Step 1: docker pull nginx:alpine fetches the image explicitly, before any container runs
- Step 2: docker images lists images already pulled
- Step 3: docker run hello-world; expected Hello from Docker!
- Step 4: docker ps shows currently running containers
- Step 5: docker ps -a shows hello-world as Exited (0)
- Step 6: docker run -d --name web-demo -p 8080:80 nginx:alpine starts a server
- Step 7: docker ps confirms web-demo running with its port mapping
- Step 8: browser at localhost:8080 shows Welcome to nginx!
- Step 9: docker stop web-demo halts the container
- Step 10: docker rm web-demo removes it; docker ps -a confirms it is gone

Visual Guidance: Numbered step strip 1 to 10, icons for terminal, container box, and browser window at the matching steps.

Speaker Notes: "This full trace is the baseline practical before the custom build in Part B. Exited (0) on hello-world is expected, not an error."

Slide 9: Running A Container

Heading: Container Lifecycle Commands

Subheading: Run Check Stop Remove

Content:

A long-running service container is started, mapped to a host port, then checked, stopped and removed through the client-daemon flow.

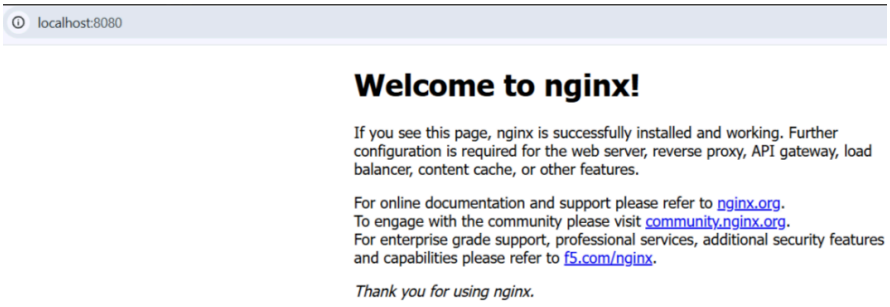
Nginx is used only to demonstrate a web application running inside a container, not as a production choice.

```
docker run -d --name web-demo -p 8080:80 nginx:alpine
```

- Line 1 — -d: run in background. --name web-demo: name the container. -p 8080:80: map host port 8080 to container port 80. nginx:alpine: the image used.
- docker images: lists cached images before running
- docker ps: expected web-demo Up, PORTS 0.0.0.0:8080->80/tcp
- Browser at port 8080: expected the Welcome to nginx! page

- docker stop web-demo: expected the container name echoed back
- docker rm web-demo: expected web-demo absent from docker ps -a

Browser Check — Step 8



Browser at localhost:8080 — "Welcome to nginx!" confirms web-demo is serving traffic

Visual Guidance: Timeline strip: run -> ps -> browser -> stop -> rm, each step labelled with its command. Only the browser-result screenshot above is currently available.

Speaker Notes: "Nginx here is only a convenient web server to prove the container is serving traffic. Removing the container frees its name but the nginx:alpine image stays cached. [Instructor note: only the browser-result screenshot (Step 8) is currently available for this slide. Steps 1, 2, 4, 5, 6, 7, 9 and 10 still need to be captured from a live run — Step 3 (hello-world) is already shown on Slide 5 and does not need a separate capture here.]"

Slide 10: Layers And Build Caching

Heading: Layers Build Caching

Subheading: Dockerfile To Layers

Content:

Each Dockerfile instruction adds one layer on top of the base image; a rebuild reuses an unchanged layer instead of redoing its work.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

- Line 1 — FROM sets the base layer.
- Line 2 — COPY adds the HTML layer.
- Line 3 — EXPOSE documents port 80, not published yet.
- Cache key: instruction plus the files it touches
- A changed instruction invalidates it and every layer after
- Stable instructions first, frequently changing ones last

Analogy : Building a house floor by floor — if the foundation and lower floors remain unchanged, you don't rebuild them when making a change to the top floor, just as Docker reuses cached layers when earlier instructions remain unchanged.

Visual Guidance: Vertical stack of layer blocks labelled FROM, COPY, EXPOSE, with a cache tick mark on the unchanged ones.

Speaker Notes: "EXPOSE only documents a port; -p at runtime actually publishes it. Interviews often ask why instruction order affects build speed."

Slide 11: Building A Custom Image

Heading: Build Run Verify

Subheading: Custom Image Trace

Content:

Part B builds a custom image from a Dockerfile, confirming the same image-container relationship already seen with pulled images.

```
cloud-docker-demo/
├── index.html
└── Dockerfile
```

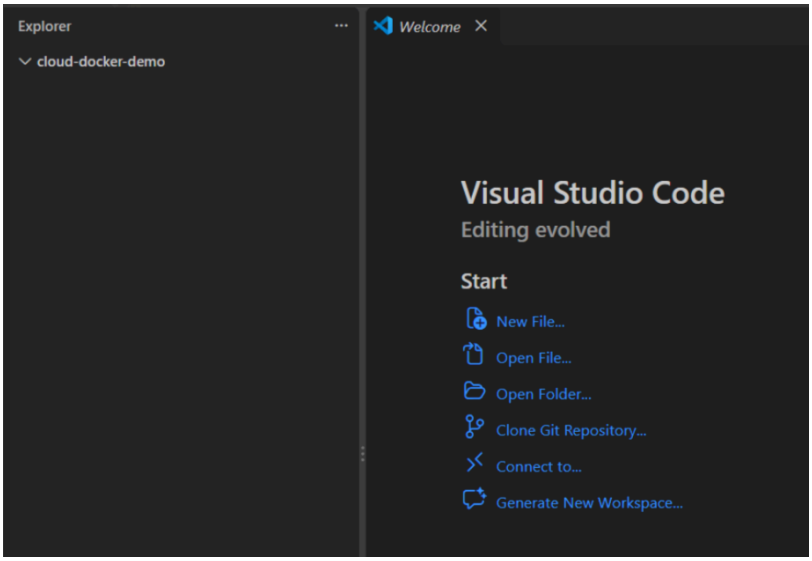
- Lines 1-3 — A project folder holding the page content and the build instructions.

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web application is running inside a Docker container.</p>
</body>
</html>
```

- Lines 6-7 — The heading and paragraph this page serves, shown by the container once built and run.
- Step 1: create folder cloud-docker-demo/ with index.html and Dockerfile
- Step 2: write index.html with the page content above

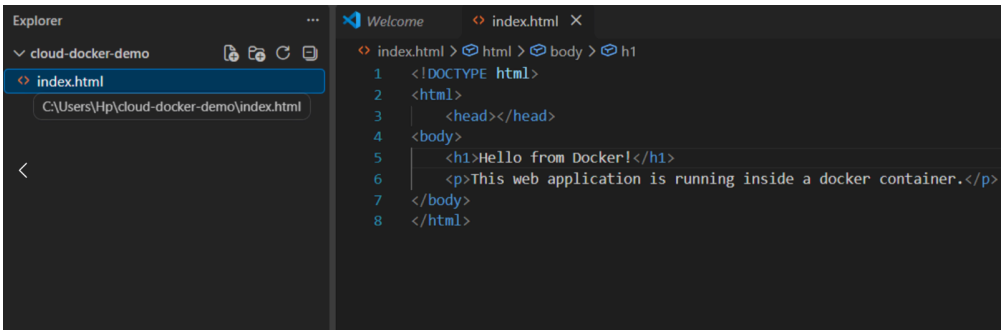
- Step 3: write Dockerfile with FROM, COPY and EXPOSE
- Step 4: docker build -t cloud-docker-demo . builds the image
- Step 5: docker images confirms cloud-docker-demo stored locally
- Step 6: docker run -d --name cloud-web -p 8080:80 cloud-docker-demo starts it
- Step 7: docker ps expected cloud-web with 0.0.0.0:8080->80/tcp
- Step 8: browser at port 8080 expected the custom page
- Step 9: docker stop cloud-web and docker rm cloud-web remove it
- Step 10: docker images still lists cloud-docker-demo after removal

Step 1 — Project Folder Created



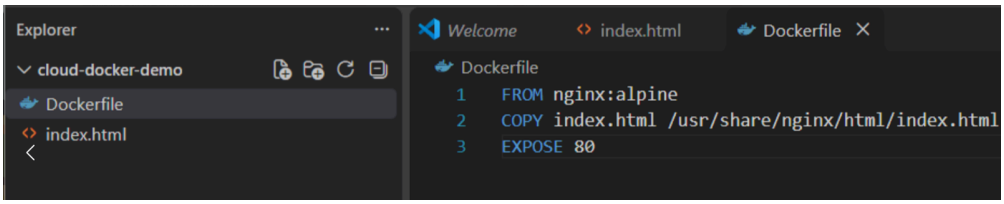
VS Code — cloud-docker-demo folder created in the file explorer

Step 2 — index.html Written



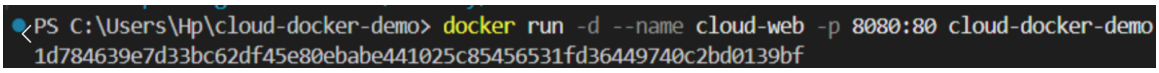
VS Code — index.html content: the custom page this container will serve

Step 3 — Dockerfile Written



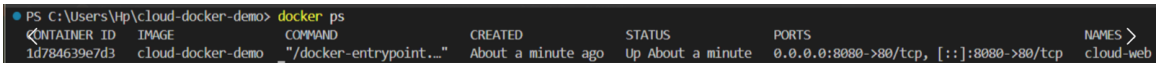
VS Code — Dockerfile content: FROM, COPY and EXPOSE directives

Step 6 — Container Started



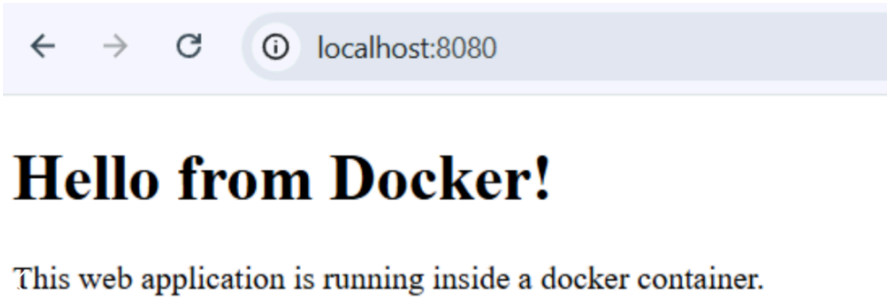
Terminal after docker run — cloud-web container ID printed

Step 7 — Port Mapping Confirmed



docker ps — cloud-web mapped to 0.0.0.0:8080->80/tcp

Step 8 — Custom Page Verified



Browser at localhost:8080 — the custom "Hello from Docker!" page

Visual Guidance: Flow strip: Dockerfile -> docker build -> image -> docker run -> container -> browser output, matching the 6 screenshots above (Steps 1, 2, 3, 6, 7, 8).

Speaker Notes: "The same build-run-remove pattern applies to any custom image. Interviews often ask which flag tags an image during build. [Instructor note: screenshots for Steps 4, 5, 9 and 10 (docker build output, docker images before and after removal, and the stop/rm confirmation) still need to be captured — not available in the current set.]"

---------- SECTION 3: 3. Containers vs VMs: Boot time, memory footprint, image size & density; isolation strength & security trade-offs ----------

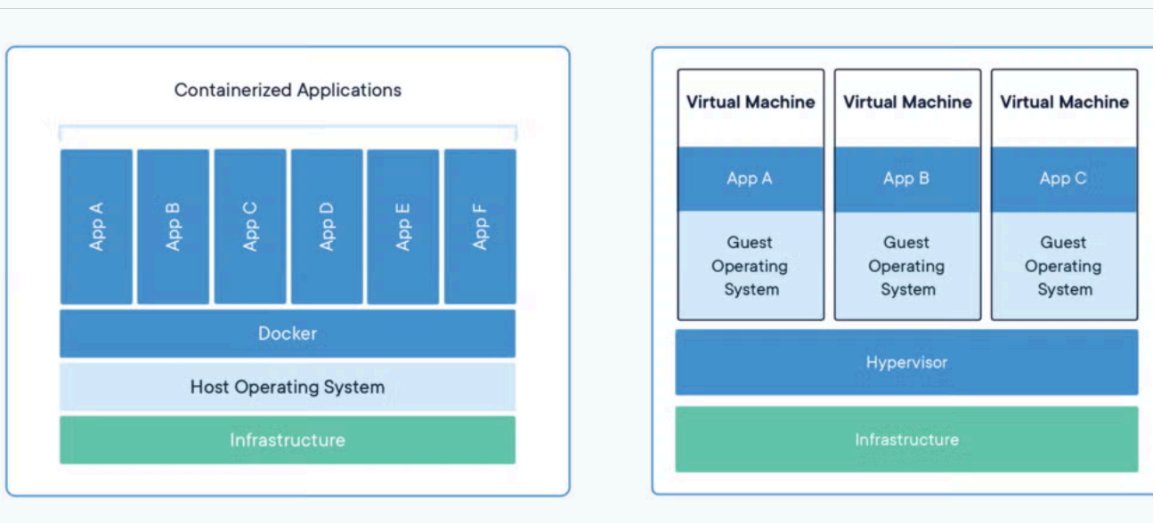
Slide 12: Comparing Containers And Virtual Machines

Heading: Four Comparison Dimensions

Subheading: Boot Memory Size Security

Content:

Containers and virtual machines isolate workloads differently, and that difference shows up in four measurable areas before any use case is chosen.



- Boot time: how quickly each starts and becomes usable
- Memory footprint: RAM consumed per workload at idle
- Image size and density: how many instances fit per host
- Isolation strength: kernel-level sharing versus hardware-level separation

- Security trade-offs: what fails if the boundary is breached

Visual Guidance: Two stacks side by side: left hardware to hypervisor to guest OS to VMs; right hardware to host OS kernel to container engine to containers.

Speaker Notes: "Set up the four comparison dimensions before any numbers appear on the following slides. Interviewers often open this exact topic by asking for these four axes directly."

Slide 13: Boot Time And Memory Footprint

Heading: Boot Time And Memory

Subheading: Startup Speed Compared

Content:

Starting a virtual machine boots a full guest operating system; starting a container launches one process on a kernel already running.

Metric	Virtual Machine	Container
Boot time	Tens of seconds (full OS boot)	Under a few seconds (process start)
Memory footprint (idle)	Several hundred MB to a few GB (guest OS overhead)	A few MB to tens of MB (app plus runtime)

- Cause: VM repeats bootloader, kernel init and service startup
- Container skips straight to launching the application process
- Lower per-instance memory lets more workloads run concurrently

Visual Guidance: Bar chart: boot time in seconds, VM bar far taller than container bar; second bar chart for idle memory, same proportion.

Speaker Notes: "Tie the boot-time gap to the OS boot sequence, not to raw hardware speed. The seconds-versus-tens-of-seconds contrast is a standard interview comparison point."

Slide 14: Image Size And Container Density

Heading: Image Size Density

Subheading: Packing Workloads Tighter

Content:

Containers fit more workloads per host by sharing the host kernel and avoiding separate guest operating systems.

Metric	Virtual Machine	Container

Image size	Several GB (full guest OS plus app)	Tens of MB to a few hundred MB (app layers only)
Typical density per host	Tens of VMs	Hundreds of containers

- Smaller images pull and deploy faster across a cluster
- Higher density lowers infrastructure cost per workload on identical hardware
- Shared base layers cut total storage across many container images

Visual Guidance: Grid diagram: one host square packed with few large VM blocks versus the same size host packed with many small container blocks.

Speaker Notes: "Density is the number behind why a host runs far more containers than VMs side by side. Expect a question on why container density beats VM density on identical hardware."

Slide 15: Isolation Strength Containers Versus VMs

Heading: Isolation Strength Compared

Subheading: Kernel Versus Hardware

Content:

A virtual machine isolates at the hardware boundary with its own kernel; a container isolates at the process boundary on one shared kernel.

- VM: separate guest kernel per instance, hypervisor enforces the boundary
- Container: namespaces separate the view, cgroups cap the resources
- Shared kernel means one kernel vulnerability can reach every container
- VM: A guest OS crash usually does not affect sibling VMs.

Visual Guidance: Boundary diagram: VM boundary drawn around hardware layer (per guest); container boundary drawn around process layer (one shared kernel line beneath all).

Speaker Notes: "This single fact explains why containers are called less isolated than VMs — shared kernel, not weaker code. Interviewers test this by asking what a kernel exploit can reach in each model."

Slide 16: Security Trade-offs Between Containers And VMs

Heading: Security Trade-offs

Subheading: Blast Radius Differs

Content:

Stronger isolation does not mean fewer attacks; it changes how far a successful attack can spread once the boundary is crossed.

- VM breakout may exploit a hypervisor vulnerability and can affect isolation between VMs
- Container breakout exploits the shared kernel: mitigated by seccomp, SELinux, non-root users
- Untrusted or multi-tenant code favours VM-grade isolation
- Trusted internal services accept container isolation for speed and density

Visual Guidance: Two circles labelled blast radius: small circle around one VM after a breakout, large circle around the whole host after a container breakout.

Speaker Notes: "Link blast radius back to the kernel-sharing fact from the isolation comparison just taught. This security-versus-density trade-off is a frequent systems-design interview question."

---------- SECTION 4: 4. Choosing Between Them: Container use cases; VM use cases; decision criteria ----------

Slide 17: Choosing Containers Or VMs

Heading: Choosing Between Them

Subheading: Use Cases And Criteria

Content:

Container and virtual machine strengths point to different workloads; matching the right one to the job is the real skill, not finding one universal winner.

- Workload type: stateless microservices vs full-stack legacy apps
- Isolation need: process-level sharing vs hardware-level separation
- OS requirement: same kernel vs mixed operating systems
- Scale and density: many light instances vs fewer, heavier ones

Visual Guidance: Diagram: a fork from one question mark — left arrow to a container icon tagged with workload types, right arrow to a VM icon tagged with workload types.

Speaker Notes: "Frame this as a workload-driven decision rather than a technology preference, since that is the real trade-off teams make daily. This exact mapping is what interviewers probe when testing container-versus-VM judgment."

Slide 18: Container Use Cases

Heading: Container Use Cases

Subheading: Where Containers Fit Best

Content:

Containers suit workloads that restart often, scale horizontally, and ship as small, repeatable units — the opposite of a slow-booting, heavyweight guest.

- Microservices: independently deployable services communicating over APIs
- CI/CD pipelines: identical images carried from build to production
- Cloud-native apps: run at scale on AWS ECS, EKS or Fargate
- Stateless, short-lived tasks: batch jobs and queue workers
- High-density hosting: many app instances packed onto one host

Visual Guidance: Row of icons: microservice mesh, CI/CD pipeline arrow, ECS/EKS logo, batch-job icon, stacked boxes for density.

Speaker Notes: "Tie each case back to the density and startup-speed advantages already established, applied here to real workload choices. Expect interview questions phrased as which workload suits a container — these five cases are the answer key."

Slide 19: Virtual Machine Use Cases

Heading: VM Use Cases

Subheading: Where VMs Fit Best

Content:

Virtual machines earn their heavier footprint when a workload needs a full, isolated operating system or hardware access the host runtime cannot provide.

- Legacy applications: tied to a specific OS version or kernel module
- Mixed operating systems: Windows and Linux guests on one host
- Strong multi-tenant isolation: separate kernels for untrusted workloads
- Kernel-level or driver work: isolated environments for kernel development and driver testing.
- Full-stack test labs: AWS EC2 instances cloning a production server exactly

Visual Guidance: Icons: Windows and Linux logos side by side, a lock for isolation, EC2 logo for the test-lab case.

Speaker Notes: "Connect each case back to the full-OS isolation and boot-time cost already established, now framed as the reason to choose a VM. A classic interview probe asks why not containerise everything — kernel dependency and tenant isolation are the answer."

Slide 20: Decision Criteria Compared

Heading: Decision Criteria Map

Subheading: Workload Fit Checklist

Content:

No single criterion decides the choice; the table weighs isolation, OS diversity, density and workload lifetime side by side.

Criterion	Favours Container	Favours VM
Isolation strength	Process-level, shared kernel	Hardware-level, separate kernel
OS diversity	Same kernel for all apps	Multiple OS types on one host
Density / footprint	Many instances per host	Fewer, heavier instances per host
Deployment and management	Frequent, consistent redeployment	Full OS control or legacy environment requirements
Tenant isolation	Lower guarantee, shared kernel	Strict guarantee for untrusted tenants

Production systems often combine both — containers scheduled onto VM-based hosts — using the VM as the security boundary and containers for density within it.

Visual Guidance: Keep the table as the slide centrepiece; below it, a small nested box showing container icons inside one VM icon to depict the hybrid pattern.

Speaker Notes: "Use this table as a quick checklist when justifying an architecture choice, since most real decisions weigh several rows together. Interview panels often ask for a one-line justification per row, which this table trains directly."

---------- SECTION 5: 5. Tooling & Registries: VirtualBox, KVM, Hyper-V & VMware; Docker, Podman & containerd; public vs private registries; tagging strategy ----------

Slide 21: Virtualization And Container Tooling Overview

Heading: Tooling Landscape

Subheading: Four Areas Mapped

Content:

Running virtual machines or containers in production means picking a hypervisor or engine, then a registry to distribute the resulting images.

- Virtualization tools: VirtualBox, KVM, Hyper-V, VMware
- Container engines: Docker, Podman, containerd
- Registries: public vs private
- Tagging: naming scheme for image versions

Visual Guidance: Two-column diagram: VM tools column, container tools column, both arrows pointing down into one shared Registry box.

Speaker Notes: "Set up these four areas before going tool by tool. Interviewers often ask for alternatives to Docker or VirtualBox by name."

Slide 22: Comparing VirtualBox, KVM, Hyper-V And VMware

Heading: VM Tooling Compared

Subheading: Four Hypervisor Products

Content:

Each virtualization product fits a different deployment, from a laptop test lab to a production data centre.

Tool	Type	Platform	Typical Use
VirtualBox	Type 2 hosted	Windows macOS Linux	Desktop testing, dev labs
KVM	Type 1, kernel module	Linux hosts	AWS Nitro, Google Cloud, OpenStack clouds
Hyper-V	Type 1 bare-metal	Windows Server, 10/11 Pro	Microsoft Azure, enterprise
VMware	Type 1 (ESXi) / Type 2 (Workstation)	Bare-metal (ESXi); Windows, Linux, macOS (Workstation/Fusion)	Enterprise data centres

Visual Guidance: Four-row table, one small vendor logo icon beside each tool name.

Speaker Notes: "Tie each product back to the Type 1 versus Type 2 split already taught. Interviewers often ask which hypervisor backs a named cloud platform."

Slide 23: Containerd And Podman Beneath Docker

Heading: Beneath Docker's Daemon

Subheading: Containerd And Podman

Content:

Docker delegates container execution to containerd. Kubernetes can use containerd through the Container Runtime Interface (CRI), without relying on Docker Engine.

- containerd: split out of Docker in 2016, now a separate project
- containerd runs the OCI Runtime Spec through runc underneath
- Podman removes the central daemon altogether; each container is a direct process
- Podman's rootless mode starts containers without any root privilege

Analogy: A city bus needs a central depot dispatching every trip, while an auto-rickshaw driver picks up a passenger directly with no depot involved — exactly as Podman starts a container directly, with no daemon standing between the command and the running process.

Visual Guidance: Diagram: Docker box with arrow down to containerd box; separate Podman box with no daemon, arrow straight to container.

Speaker Notes: "Containerd living inside Docker, and now also standalone, is a common interview trick question. Note that Podman's daemonless design is the security motivation, not a performance one."

Slide 24: Docker Vs Podman Vs Containerd

Heading: Engines Compared

Subheading: Architecture And Role

Content:

The three tools sit at different layers of one stack, and that layering decides which one fits a given deployment.

Aspect	Docker	Podman	containerd
Daemon required	Yes, dockerd	No	Yes, lower-level service
Rootless by default	No	Yes, when run as a normal user	No, root-level service
Native CLI	Yes, full-featured	Yes, Docker-compatible	Minimal (ctr, for debugging); nerdctl for Docker-like use
Primary role	Dev workstation, CI	Secure, rootless hosts	Kubernetes CRI runtime (EKS, GKE)

Visual Guidance: Four-row comparison table, highlight the Primary Role row in a contrasting colour.

Speaker Notes: "This table is the one most asked about in container-tooling interviews. Stress that containerd, not Docker, is the runtime a Kubernetes node actually calls."

Slide 25: Public Vs Private Container Registries

Heading: Public Vs Private

Subheading: Image Registries

Content:

A registry stores and distributes container images; which kind is used decides who is able to pull a given image.

- Public registry: Docker Hub, where public images can generally be pulled by anyone
- Private registry: Amazon ECR, Azure Container Registry, Google Artifact Registry
- Self-hosted option: Harbor, run inside an organisation's own network
- Chosen for security, compliance or protecting proprietary images

Analogy: A public registry is like a library open to any visitor, while a private registry is like an office cupboard that opens only with an employee's key — exactly as a private registry demands authentication before an image can be pulled.

Visual Guidance: Split diagram: public registry with an open padlock icon, private registry with a closed padlock, both feeding a docker pull arrow.

Speaker Notes: "Distinguish a registry, the service, from a repository, one image's set of tags, if asked in an interview. Private registries are the default choice for any proprietary production image."

Slide 26: Tagging Strategy For Container Images

Heading: Tagging Strategy

Subheading: Naming Image Versions

Content:

Image tags provide human-readable names for versions, while digests identify exact image content for reliable redeployment.

- Semantic versioning: major.minor.patch, for example app:2.3.1
- latest tag: moves with every push, unsafe for production rollback

- Immutable tag: git-commit hash, for example app:a1b2c3d, never overwritten
- Environment tags: app:staging, app:prod mark the pipeline stage

Visual Guidance: Timeline of three pushes to one repository, each landing a different immutable tag, with the latest pointer jumping between them.

Speaker Notes: "Flag latest as the most common cause of a failed production rollback when this comes up in interviews. Immutable, hash-based tags are what real CI/CD pipelines rely on for traceability."

Heads-Up: Start Your AWS Account Setup Now

Why now: AWS account setup is required before Practicals. Account verification may take a day or more. A free-tier account is sufficient.

Key Takeaways

1. Container Fundamentals: Process-level isolation; Linux namespaces & control groups (cgroups); union filesystem & layered images
2. Docker Architecture: Client, daemon, images, containers & registries; image vs container; layers & build caching
3. Containers vs VMs: Boot time, memory footprint, image size & density; isolation strength & security trade-offs
4. Choosing Between Them: Container use cases; VM use cases; decision criteria
5. Tooling & Registries: VirtualBox, KVM, Hyper-V & VMware; Docker, Podman & containerd; public vs private registries; tagging strategy

Upcoming Session : Cloud Networking & VPC

Thank You | All the Best